

FUNDAMENTOS DE SOFTWARE — ESTACIÓN FINAL

Quantum Core: CRUD de Transacciones full-stack

Cómo funciona un sistema de tres capas —React, Flask y MySQL en Docker— y qué mejoras se hicieron sobre el proyecto base para que el usuario trabaje con más claridad y seguridad.

React + Vite

Flask

Prisma

MySQL · Docker

Javier Berrocal

Ingeniería de Sistemas

1

Tres procesos independientes, un solo contrato

- **Frontend** (React, puerto 5173): es lo que ve el usuario. Pide y envía datos con `fetch()`.
- **Backend** (Flask, puerto 5000): expone la API REST `/api/transacciones` y siempre responde en JSON.
- **Base de datos** (MySQL en Docker, puerto 3306): el volumen conserva los datos aunque se borre el contenedor.
- Cada capa se puede reiniciar o cambiar sin tocar las otras: lo único que las une es el contrato de la API.



El navegador nunca habla con la base de datos: solo con la API. Si mañana hubiera una app móvil, el backend no cambiaría.

routes → controllers → models

- **routes** recibe la petición HTTP y la entrega al controller; devuelve `jsonify(data), status`.
- **controllers** valida, decide y elige el código HTTP (200, 201, 400, 404, 409, 500).
- **models** es la única capa que habla con la base de datos, a través de Prisma.

MÉTODO	RUTA	QUÉ HACE
GET	/	Lista todas
GET	/resumen	Totales por tipo
GET	/ <id>	Una transacción
POST	/	Crea
PUT	/ <id>	Actualiza
DELETE	/ <id>	Elimina

Cada capa tiene una sola responsabilidad (SRP): si cambia la base de datos, solo se toca el model.

BACKEND/routes/transaccion_routes.py

```
8 @transaccion_bp.route("/", methods=["GET"])
9 def listar():
10     data, status = controller.listar_transacciones()
11     return jsonify(data), status
12
13
14 @transaccion_bp.route("/resumen", methods=["GET"])
15 def resumen():
16     data, status = controller.obtener_resumen()
17     return jsonify(data), status
18
19
20 @transaccion_bp.route("/<int:id>", methods=["GET"])
21 def obtener(id):
22     data, status = controller.obtener_transaccion(id)
23     return jsonify(data), status
24
25
28 @transaccion_bp.route("/", methods=["POST"])
29 def crear():
30     data, status = controller.crear_transaccion(request.get_json(silent=True))
31     return jsonify(data), status
```

El modelo Transaccion y su capa de datos

- `schema.prisma` define la tabla: `id` autoincremental y `codigo` `@unique`, así no hay dos transacciones con el mismo código.
- El model expone una función de una línea por operación; Prisma la traduce a SQL para MySQL.
- MySQL corre en el contenedor Docker `empresa`: `docker start empresa`.

BACKEND/schema.prisma

```
12 model Transaccion {
13   id      Int    @id @default(autoincrement())
14   codigo  String @unique
15   tipo    String
16   monto   Int
17   impacto Float
18 }
```

BACKEND/models/transaccion_model.py

```
4 def obtener_todas():
5     return db.transaccion.find_many(order={"id": "desc"})
6
8 def obtener_por_id(id):
9     return db.transaccion.find_unique(where={"id": id})
10
12 def obtener_por_codigo(codigo):
13     return db.transaccion.find_unique(where={"codigo": codigo})
14
16 def crear(datos):
17     return db.transaccion.create(data=datos)
18
20 def actualizar(id, datos):
21     return db.transaccion.update(where={"id": id}, data=datos)
22
24 def eliminar(id):
25     return db.transaccion.delete(where={"id": id})
```

Persistencia real: al recargar la página o reiniciar el backend, los datos siguen ahí porque viven en MySQL, no en memoria.

4

FLUJO PRINCIPAL

Crear una transacción, de punta a punta

- Una acción del usuario recorre las tres capas y vuelve con una respuesta JSON.
- Cada función del controller está dentro de `try / except` : un fallo devuelve un error controlado y el servidor no se cae.
- En el frontend, `try / catch` atrapa ese error y lo muestra en pantalla.
- Editar (`PUT`) y eliminar (`DELETE`) siguen el mismo camino.

1 Formulario
El usuario llena código, tipo, monto e impacto y pulsa *Crear*.
REACT

2 `peticion()`
`POST /api/transacciones/` con el cuerpo en JSON.
API/TRANSACCIONES.JS

3 Route
`crear()` lee el JSON con `get_json(silent=True)` .
ROUTES

4 `validar_datos()`
Revisa y normaliza; si algo falla responde **400**.
CONTROLLERS

5 `model.crear()`
Prisma inserta la fila en MySQL.
MODELS · DOCKER

6 201 + recarga
`cargarDatos()` refresca la tabla y las tarjetas.
REACT

Es el mismo encapsulamiento del *setter* de POO: ningún dato llega a la base sin pasar antes por la validación del controller.

SEGUNDA PARTE

Antes vs. después

Once mejoras pequeñas sobre el proyecto base del profesor, sin cambiar la arquitectura `routes → controllers → models`.

11

MEJORAS

6

ARCHIVOS MODIFICADOS

2

ARCHIVOS NUEVOS

0

CAMBIOS AL ESQUEMA

La misma app, más fácil de leer

- **Tarjetas de resumen** (nuevo endpoint `/resumen`): total, créditos, débitos y transferencias de un vistazo.
- **Montos en pesos colombianos**: `1500000` pasa a `$ 1.500.000`, alineados a la derecha.
- **Un color por tipo**: antes débito y transferencia se veían iguales; ahora verde, rojo y azul.
- **Impacto negativo en rojo** y **buscador + filtro** encima de la tabla.

El usuario ya no tiene que contar ni leer cifras sin formato: la interfaz le resume la información.



● ANTES · TABLA ORIGINAL

Código	Tipo	Monto	Impacto	Acciones
T006	TRANSFERENCIA	85000	0.35	Editar Eliminar
T005	DEBITO	120000	-1.2	Editar Eliminar

● DESPUÉS · TARJETAS DE RESUMEN

TOTAL

\$ 2.415.000

6 transacciones

CRÉDITOS

\$ 1.950.000

2 registros

DÉBITOS

\$ 180.000

2 registros

TRANSFERENCIAS

\$ 285.000

2 registros

● DESPUÉS · TABLA CON FORMATO Y COLORES

Código	Tipo	Monto	Impacto	Acciones
T006	TRANSFERENCIA	\$ 85.000	0,35	Editar Eliminar
T005	DEBITO	\$ 120.000	-1,20	Editar Eliminar
T004	CREDITO	\$ 450.000	2,75	Editar Eliminar

De una regla suelta a `validar_datos()`

ANTES · REPETIDO EN CREAR Y ACTUALIZAR

transaccion_controller.py · original

```
18 if datos.get("monto", 0) < 0:
19     return {"error": "El monto no puede ser negativo"}, 400
```

- Solo revisaba que el monto no fuera negativo; aceptaba `tipo="HOLA"`, campos vacíos o un `id` extra.
- **Ahora**, una sola función (DRY) revisa los campos obligatorios, normaliza a mayúsculas, exige un tipo válido y un monto entero mayor que cero.
- Copia solo los 4 campos permitidos (lista blanca).

Para el usuario: si se equivoca, recibe un mensaje concreto ("El monto debe ser mayor que cero") en lugar de un error técnico.

transaccion_controller.py · validar_datos()

```
19 if not isinstance(datos, dict):
20     return None, "El cuerpo de la petición debe ser un JSON válido"
21
22 faltantes = [campo for campo in CAMPOS_OBLIGATORIOS if datos.get(campo) in (None, "")]
23 if faltantes:
24     return None, f"Faltan campos obligatorios: {' '.join(faltantes)}"
25
26 codigo = str(datos["codigo"]).strip().upper()
27 tipo = str(datos["tipo"]).strip().upper()
28 monto = datos["monto"]
29 impacto = datos["impacto"]
30
31 if not codigo:
32     return None, "El código no puede estar vacío"
33 if tipo not in TIPOS_VALIDOS:
34     return None, f"Tipo inválido. Usa uno de: {' '.join(TIPOS_VALIDOS)}"
35 if not _es_numero(monto) or int(monto) != monto:
36     return None, "El monto debe ser un número entero"
37 if monto <= 0:
38     return None, "El monto debe ser mayor que cero"
39 if not _es_numero(impacto):
40     return None, "El impacto debe ser un número"
```

SITUACIÓN	ANTES	AHORA
Datos inválidos	400	400 + mensaje claro
Id que no existe	400 (Prisma)	404 No encontrada
Código repetido	400 (Prisma)	409 Conflict
Falla interna	400 + <code>str(e)</code>	500 genérico
Ruta mal escrita	Página HTML	JSON 404 / 405

- Antes de guardar, el controller consulta `obtener_por_codigo()` ; al editar solo hay conflicto si el código es de **otra** transacción.
- El detalle técnico se imprime en la consola del servidor, no se envía al navegador.

Enviar `str(e)` al cliente expone detalles de la base de datos; ahora el usuario ve qué pasó y el servidor guarda el resto.

● ANTES · CÓDIGO T001 REPETIDO

Código

t001

Tipo

CREDITO

Monto

50000

Impacto

0.5

Unique constraint failed on the constraint: `Transaccion\_codigo\_key`

Crear transacción

● DESPUÉS · MISMO CASO

Nueva transacción

Código

T001

Tipo

CREDITO

Monto

50000

Impacto

0.5

Ya existe una transacción con el código T001

Crear transacción

La app avisa en vez de fallar en silencio

- Antes, cada llamada repetía `fetch` + `.json()` y nunca revisaba errores: con el backend apagado la tabla quedaba vacía, sin explicación.
- Ahora una sola función `peticion()` convierte la falta de conexión y cualquier respuesta no 2xx en un `Error` legible.

ESTADO	QUÉ VE AHORA EL USUARIO
Cargando	"Cargando transacciones..."
Lista vacía	"Aún no hay transacciones..." o "Ninguna coincide con el filtro"
Enviando	Botón "Guardando..." deshabilitado (evita envíos dobles)
Éxito	Mensaje verde que desaparece a los 3 s
Error	Mensaje rojo con el texto exacto del backend

El usuario siempre sabe en qué estado está la aplicación: cargando, vacía, guardando, correcta o con un problema que puede resolver.



● ANTES · BACKEND APAGADO: TABLA VACÍA, SIN AVISO

Monto

Impacto

Crear transacción

Código	Tipo	Monto	Impacto	Acciones
--------	------	-------	---------	----------

● DESPUÉS · MISMO CASO: LA APP EXPLICA QUÉ PASA

No se pudo conectar con el servidor. ¿Está encendido el backend?

Crear transacción

Buscar por código...

Todos los tipos

Código	Tipo	Monto	Impacto	Acciones
Aún no hay transacciones. Crea la primera con el formulario.				

Menos errores irreversibles

- Antes, un clic en *Eliminar* borraba el registro al instante.
- Ahora la fila cambia a ¿Eliminar? Sí / No. El estado `confirmandoId` guarda qué fila pide confirmación, sin la ventana nativa `confirm()`.
- **Modo edición visible:** el formulario tiene borde ámbar y el título *Editando T004*, y la fila que se edita queda resaltada.
- El campo código se convierte a mayúsculas mientras se escribe.

● CONFIRMACIÓN AL ELIMINAR

Código	Tipo	Monto	Impacto	Acciones
T006	TRANSFERENCIA	\$ 85.000	0,35	Editar Eliminar
T005	DEBITO	\$ 120.000	-1,20	¿Eliminar? Si No
T004	CREDITO	\$ 450.000	2,75	Editar Eliminar
T003	DEBITO	\$ 60.000	-0,40	Editar Eliminar
T002	TRANSFERENCIA	\$ 200.000	0,90	Editar Eliminar
T001	CREDITO	\$ 1.500.000	5,10	Editar Eliminar

● MODO EDICIÓN RESALTADO

Editando T004

Código

T004

Tipo

CREDITO

Monto

450000

Impacto

2,75

Guardar cambios

Cancelar

Un clic accidental ya no cuesta un dato, y el usuario nunca duda de si está creando o editando.

Encontrar una transacción en segundos

- Campo de texto para buscar por código y selector para filtrar por tipo.
- La lista visible se **calcula** en cada render a partir del estado; no se guarda una segunda copia de los datos.
- Una fila se muestra si (el filtro es *TODOS* o coincide el tipo) **y** el código contiene el texto buscado.
- Los montos se muestran en pesos colombianos con `Intl.NumberFormat("es-CO")`.

Transacciones.jsx

```
132 const transaccionesVisibles = transacciones.filter(  
133   (t) =>  
134     (filtroTipo === "TODOS" || t.tipo === filtroTipo) &&  
135     t.codigo.toLowerCase().includes(busqueda.trim().toLowerCase())  
136 );
```

● BÚSQUEDA "T00" + FILTRO DEBITO

Crear transacción

Código	Tipo	Monto	Impacto	Acciones
T005	DEBITO	\$ 120.000	-1,20	Editar Eliminar
T003	DEBITO	\$ 60.000	-0,40	Editar Eliminar

Con muchos registros, el usuario no tiene que recorrer toda la tabla: escribe, filtra y ve solo lo que necesita.

Menos repetición, componentes con una sola tarea

- `peticion()` centraliza todas las llamadas: las funciones públicas quedaron de una línea cada una.
- `TarjetasResumen.jsx` es un componente aparte: la página solo le pasa el objeto `resumen`.
- `utils/formato.js` usa `Intl.NumberFormat("es-CO")` para moneda y decimales.
- Constantes `TIPOS`, `ESTILO_TIPO` y `CLASE_INPUT` en lugar de clases y opciones copiadas.
- Diseño adaptable: 1 columna en celular, 2 en pantallas grandes, y la tabla con scroll horizontal.

api/transacciones.js

```
5  async function peticion(ruta, opciones = {}) {
6    let respuesta;
7    try {
8      respuesta = await fetch(`${BASE_URL}${ruta}`, {
9        headers: { "Content-Type": "application/json" },
10       ...opciones,
11     });
12   } catch {
13     throw new Error("No se pudo conectar con el servidor. ¿Está encendido el backend?");
14   }
15
16   const datos = await respuesta.json().catch(() => ({}));
17   if (!respuesta.ok) {
18     throw new Error(datos.error || `Error ${respuesta.status}`);
19   }
20   return datos;
21 }
22
23 export function obtenerTransacciones() {
24   return peticion("/");
25 }
26
27 :
31 export function crearTransaccion(datos) {
32   return peticion("/", { method: "POST", body: JSON.stringify(datos) });
33 }
```

Un código más ordenado también mejora la experiencia: es más fácil de mantener y cada error se maneja igual en toda la app.

	ANTES, EL USUARIO...	AHORA, EL USUARIO...
Resumen	Contaba filas y sumaba montos a mano	✓ Ve totales por tipo en 4 tarjetas
Formato	Leía 15000000 y -1.2	✓ Lee \$ 1.500.000 y -1,20 en rojo
Errores	Veía mensajes de Prisma o nada	✓ Ve qué falló y cómo corregirlo
Eliminar	Perdía datos con un clic accidental	✓ Confirma con Sí / No
Editar	No sabía si estaba creando o editando	✓ Ve el formulario y la fila resaltados
Buscar	Recorría toda la tabla	✓ Filtra por código y tipo
Estados	Pantalla vacía sin explicación	✓ Mensajes de carga, éxito y error

En síntesis: las mejoras no cambian qué hace el sistema (sigue siendo un CRUD de tres capas), sino cómo se siente usarlo: más claro, más seguro y con respuestas que se entienden.

CIERRE · DEMO EN VIVO

Lo que demuestra el proyecto

POO · encapsulamiento

validar_datos() en el controller

Persistencia

Prisma + MySQL en Docker

try / except

Respuestas 400 · 404 · 409 · 500

JSON

Toda respuesta de la API

Arquitectura en capas

routes → controllers → models

Git / GitHub

Historial del repositorio

Para la demo: `docker start empresa` → `python app.py` → `npm run dev` → crear, editar, intentar un monto inválido, eliminar y recargar.